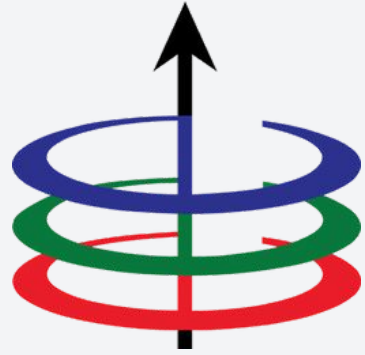




Lecture 1B

Pointers, Memory Management

EEE 121 2s2526

Steven S. Sison

- Computer Memory
- Introduction to Pointers
- Memory Management with Pointers

- **Computer Memory**
- Introduction to Pointers
- Memory Management with Pointers

Computer Memory

- Computers use memory to store information.
 - Every variable uses the memory to store its value.
 - Variable declarations => memory is reserved.
- Each data stored in memory has a corresponding address.
- Each memory address corresponds to 1 byte or 8-bit data.
 - If four contiguous memory addresses were reserved, you can store 4 bytes (32-bit) data.
- The **Operating System or the Kernel** controls where data is stored.

Memory Address	Physical Memory
0x40000000	iVar1
0x40000001	
0x40000002	
0x40000003	
0x40000004	dVar2
0x40000005	
0x40000006	
0x40000007	
0x40000008	
0x40000009	
0x4000000A	
0x4000000B	
0x4000000C	aVar3
0x4000000D	
0x4000000E	
0x4000000F	
0x40000010	

Computer Memory

For example:

```
int iVar1;    /* Reserves 4 bytes */
```

Data Type	Meaning	Size
int	integer	2/4 bytes (16/32-bit)
float	floating-point	4 bytes (32-bit)
double	floating-point	8 bytes (64-bit)
char	single character	1 byte (8-bit)
wchar_t	wide character	2 bytes (16-bit)
bool	boolean	1 byte (8-bit)
void	empty	0

4 bytes = 4
contiguous
memory
locations

Memory Address	Physical Memory
0x40000000 0x40000001 0x40000002 0x40000003	iVar1
0x40000004 0x40000005 0x40000006 0x40000007 0x40000008 0x40000009 0x4000000A 0x4000000B	dVar2
0x4000000C 0x4000000D 0x4000000E 0x4000000F 0x40000010	aVar3

Computer Memory

For example:

```
double dVar2; /* Reserves 8 bytes */
```

Data Type	Meaning	Size
int	integer	2/4 bytes (16/32-bit)
float	floating-point	4 bytes (32-bit)
double	floating-point	8 bytes (64-bit)
char	single character	1 byte (8-bit)
wchar_t	wide character	2 bytes (16-bit)
bool	boolean	1 byte (8-bit)
void	empty	0

8 bytes = 8
contiguous
memory
locations

Memory Address	Physical Memory
0x40000000	iVar1
0x40000001	
0x40000002	
0x40000003	
0x40000004	dVar2
0x40000005	
0x40000006	
0x40000007	
0x40000008	
0x40000009	
0x4000000A	
0x4000000B	
0x4000000C	aVar3
0x4000000D	
0x4000000E	
0x4000000F	
0x40000010	

Computer Memory

For example:

```
char aVar3[5];  /* Reserves 5 bytes */
```

Data Type	Meaning	Size
int	integer	2/4 bytes (16/32-bit)
float	floating-point	4 bytes (32-bit)
double	floating-point	8 bytes (64-bit)
char	single character	1 byte (8-bit)
wchar_t	wide character	2 bytes (16-bit)
bool	boolean	1 byte (8-bit)
void	empty	0

5*1 bytes = 5
contiguous
memory
locations

Memory Address	Physical Memory
0x40000000	iVar1
0x40000001	
0x40000002	
0x40000003	
0x40000004	dVar2
0x40000005	
0x40000006	
0x40000007	
0x40000008	
0x40000009	
0x4000000A	aVar3
0x4000000B	
0x4000000C	
0x4000000D	
0x4000000E	
0x4000000F	
0x40000010	

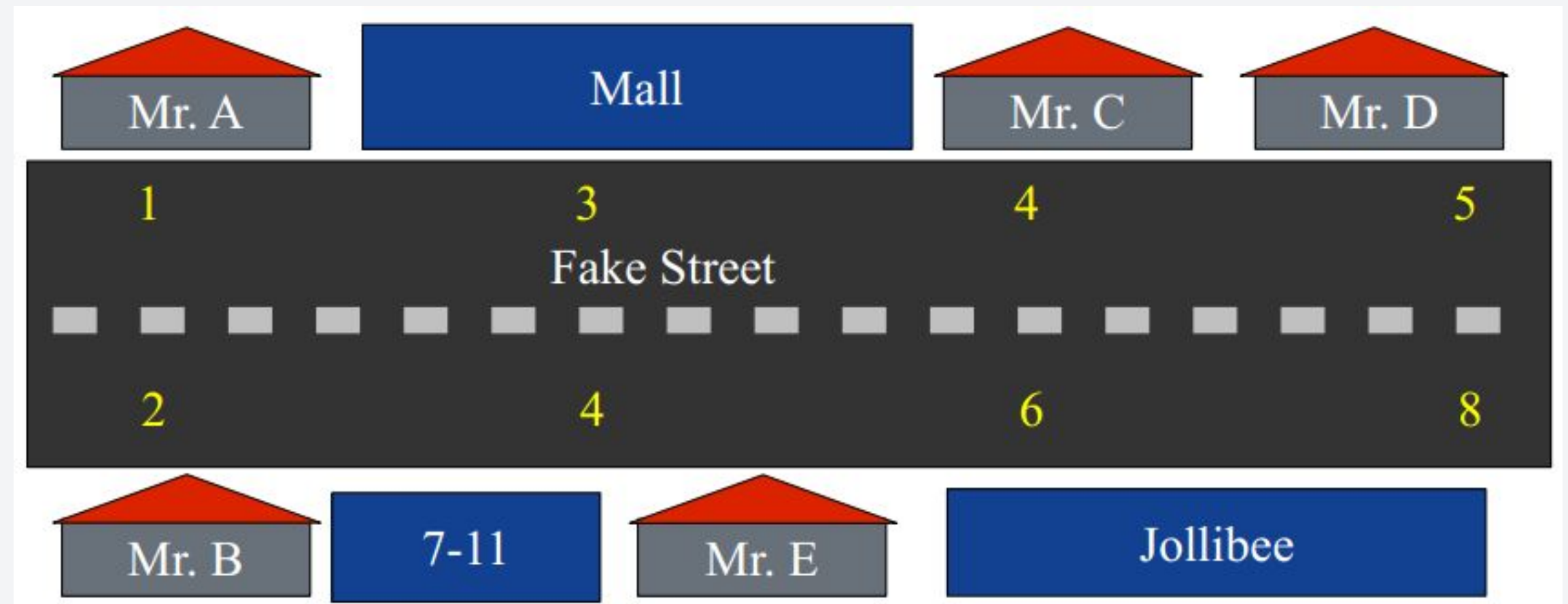
Computer Memory Analagy - House Address

In C++, it is possible to access data directly from memory through its address. So **what is the purpose of using variables?**

Computer Memory Analagy - House Address

In C++, it is possible to access data directly from memory through its address. So **what is the purpose of using variables?**

We can think of memory addresses as our street address.



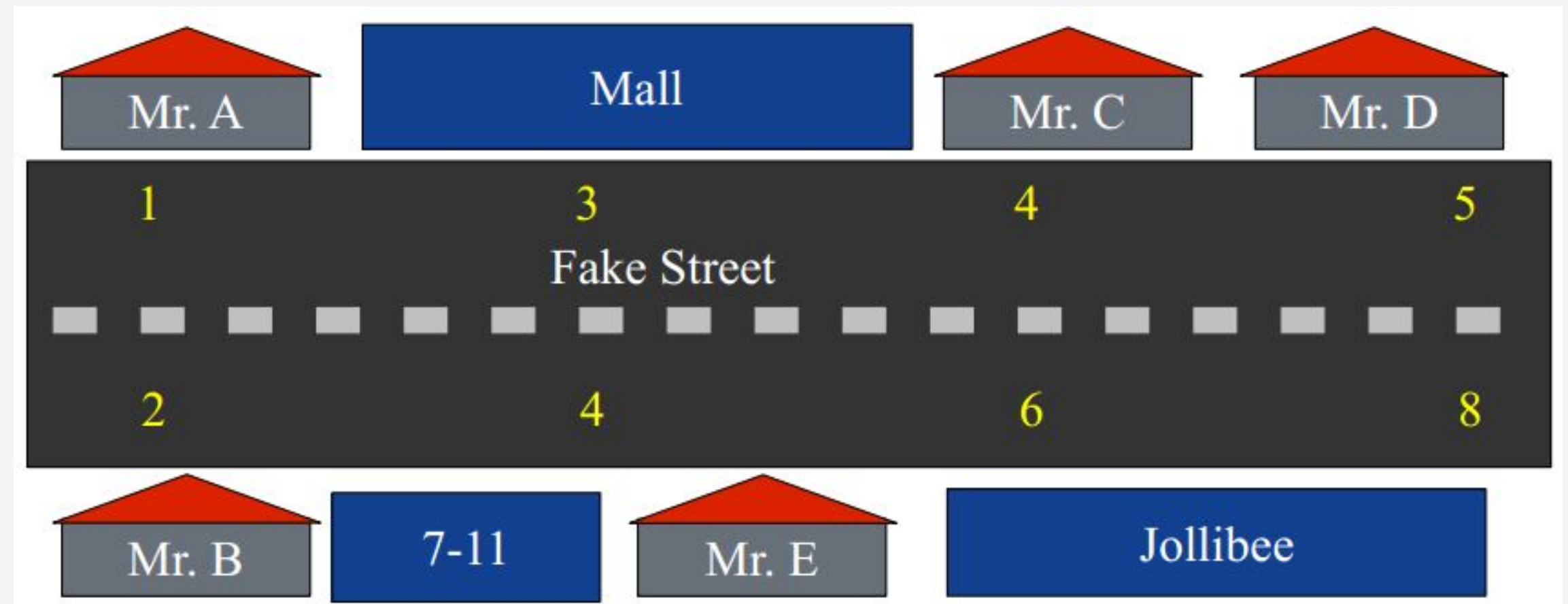
Computer Memory Analagy - House Address

In C++, it is possible to access data directly from memory through its address. So **what is the purpose of using variables?**

We can think of memory addresses as our street address.

A person living in a street has:

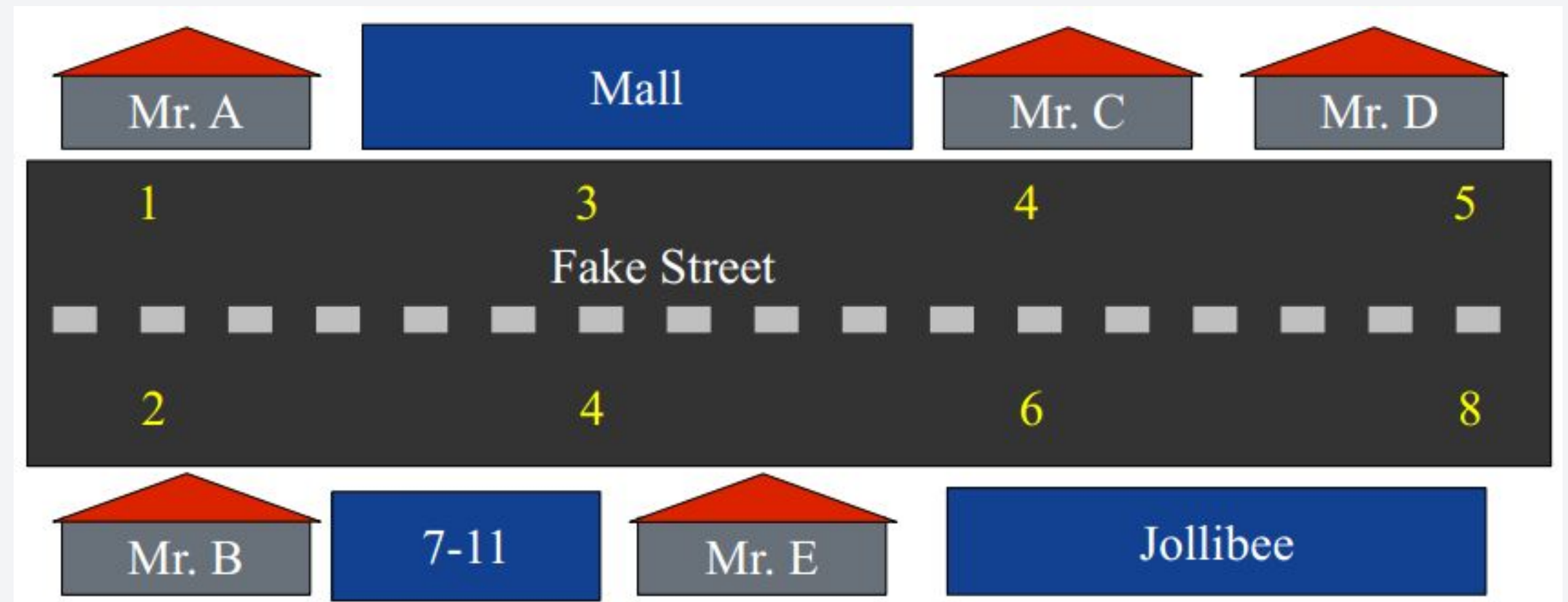
- Name of the bldg.
- Exact street address



Computer Memory Analagy - House Address

When going to a place,
its easier to associate the
name of the place rather
than its address.

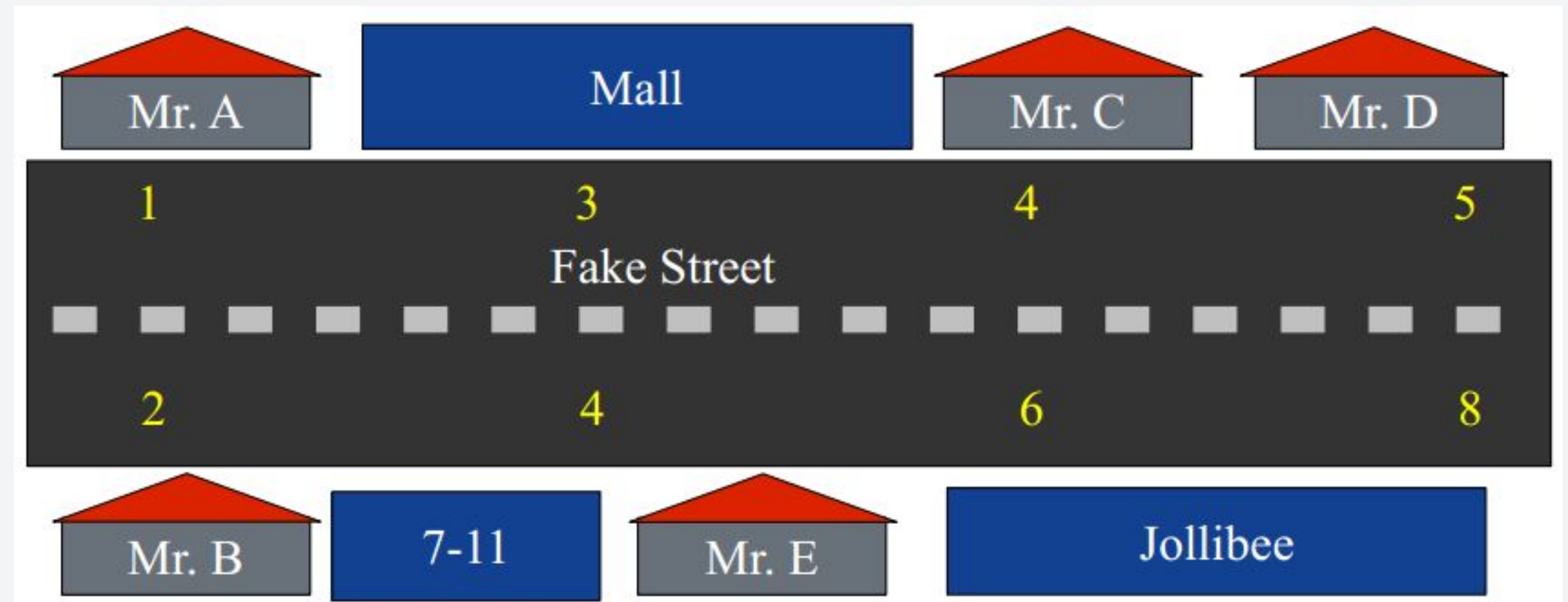
Its easier to remember
the Mall rather than #3
Fake Street.



Computer Memory Analagy - House Address

Similar to a computer memory, its easier to remember iVar instead of 0x400000000.

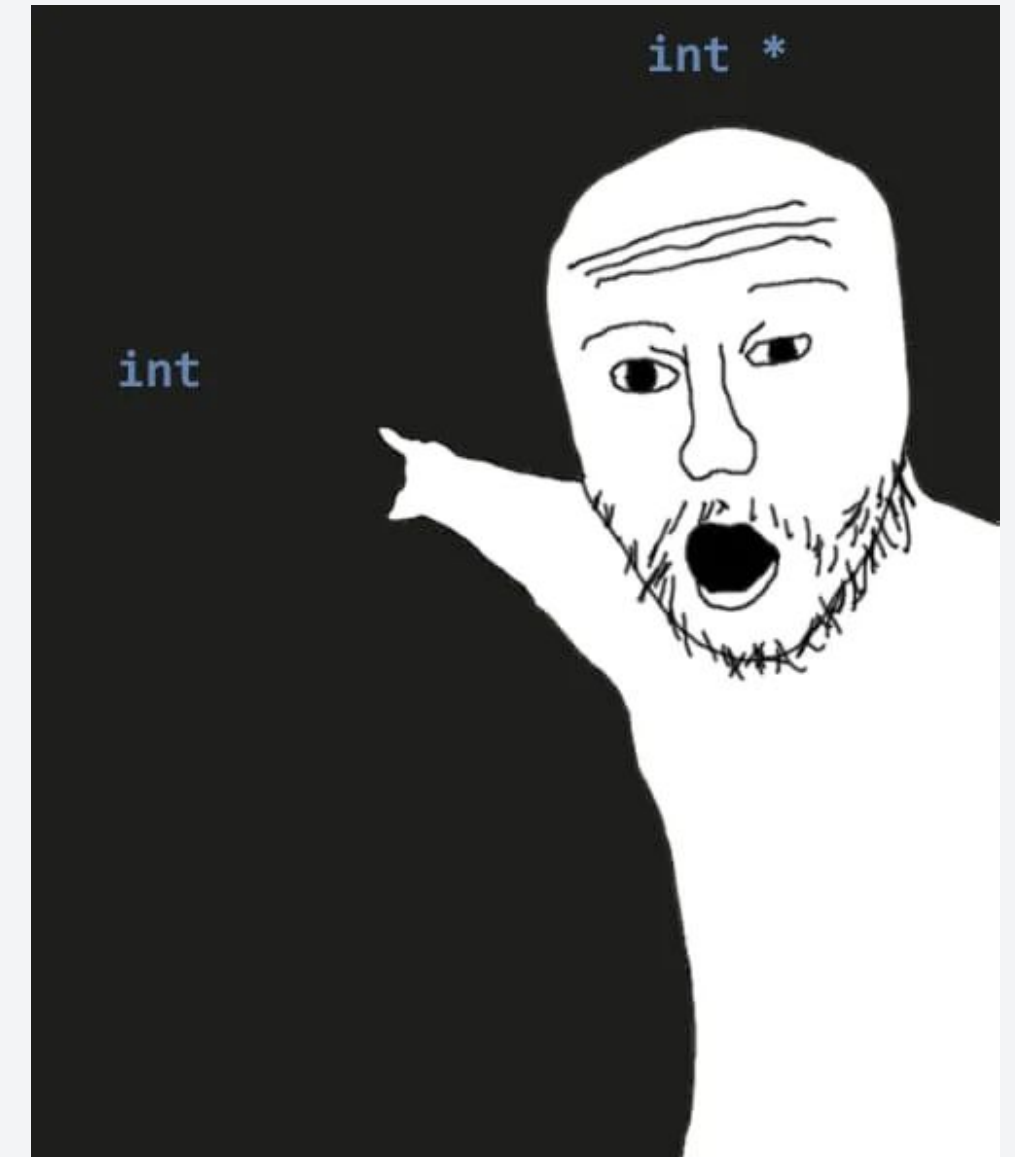
Nevertheless, C++ allows us to directly access memory through their address.



- Computer Memory
- **Introduction to Pointers**
- Memory Management with Pointers

What and why do we have pointers?

- Pointers = variables for storing memory addresses

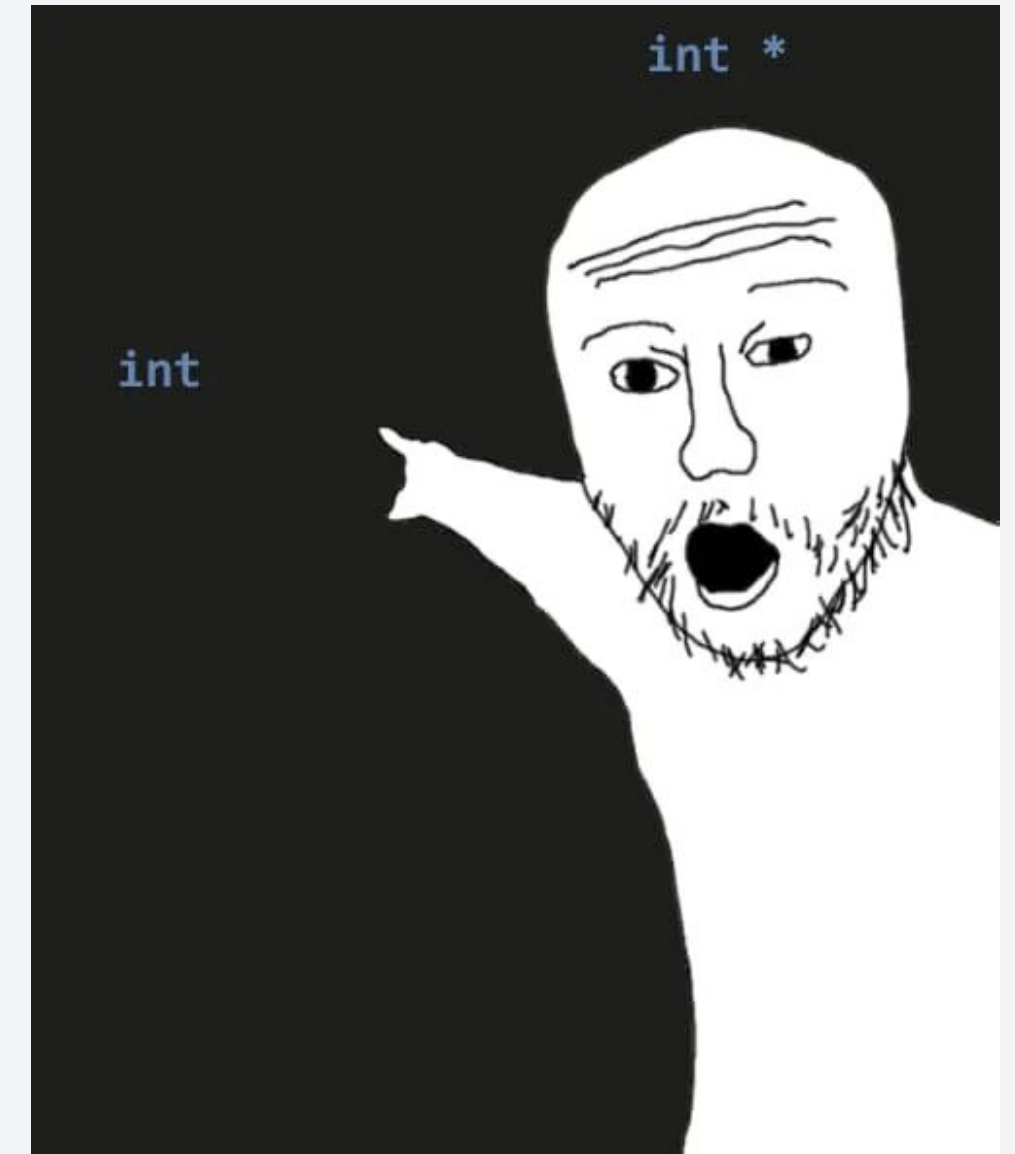


What and why do we have pointers?

- Pointers = variables for storing memory addresses

```
int m = 5;      // m stores an int
int* ptr;       // ptr stores a memory address

ptr = &m;       // store "address of" m in ptr
*ptr = 10;      // Updates the value at address
```



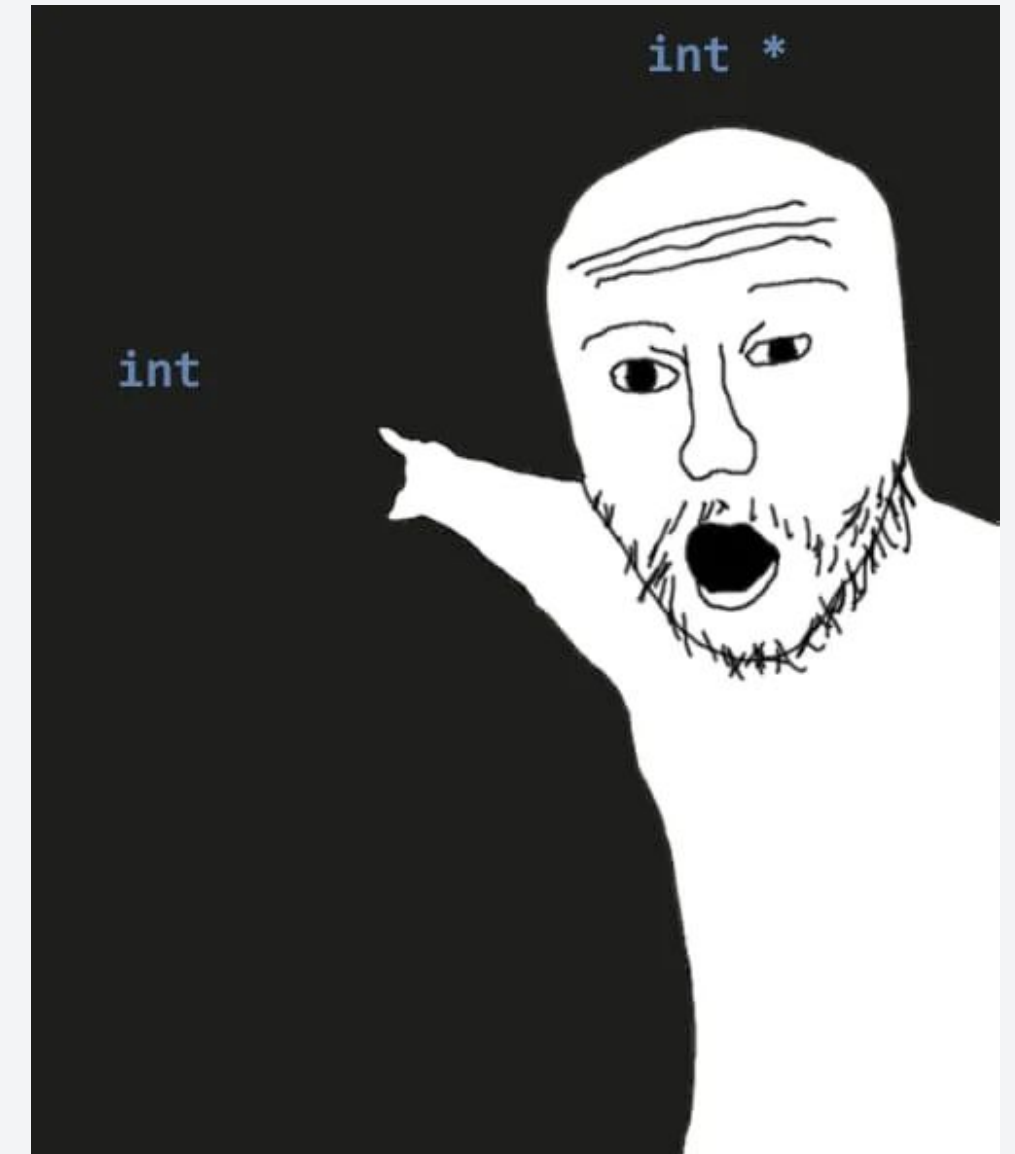
What and why do we have pointers?

- Pointers = variables for storing memory addresses

```
int m = 5;    // m stores an int
int* ptr;     // ptr stores a memory address

ptr = &m;     // store "address of" m in ptr
*ptr = 10;    // Updates the value at address
```

Variable	m	ptr
Value	5	0x00000000
Address	0x00000000	0x00000004



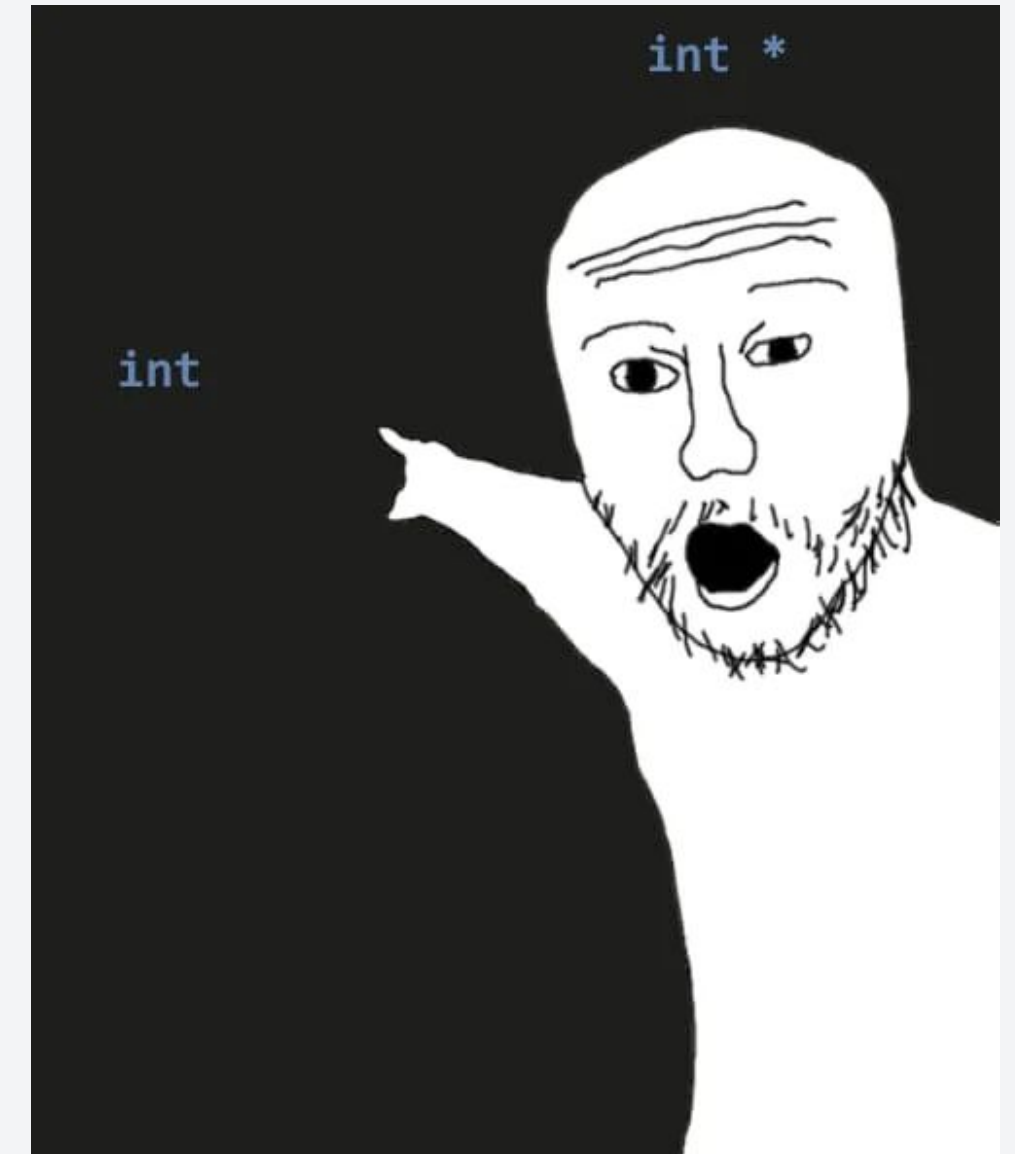
What and why do we have pointers?

- Pointers = variables for storing memory addresses

```
int m = 5;    // m stores an int
int* ptr;     // ptr stores a memory address

ptr = &m;     // store "address of" m in ptr
*ptr = 10;    // Updates the value at address
```

Variable	m	ptr	- m is stored at 0x00000000 - m contains 5 as its value
Value	5	0x00000000	
Address	0x00000000	0x00000004	



What and why do we have pointers?

- Pointers = variables for storing memory addresses

```
int m = 5;    // m stores an int
int* ptr;     // ptr stores a memory address

ptr = &m;     // store "address of" m in ptr
*ptr = 10;    // Updates the value at address
```

Variable

m

ptr

Value

5

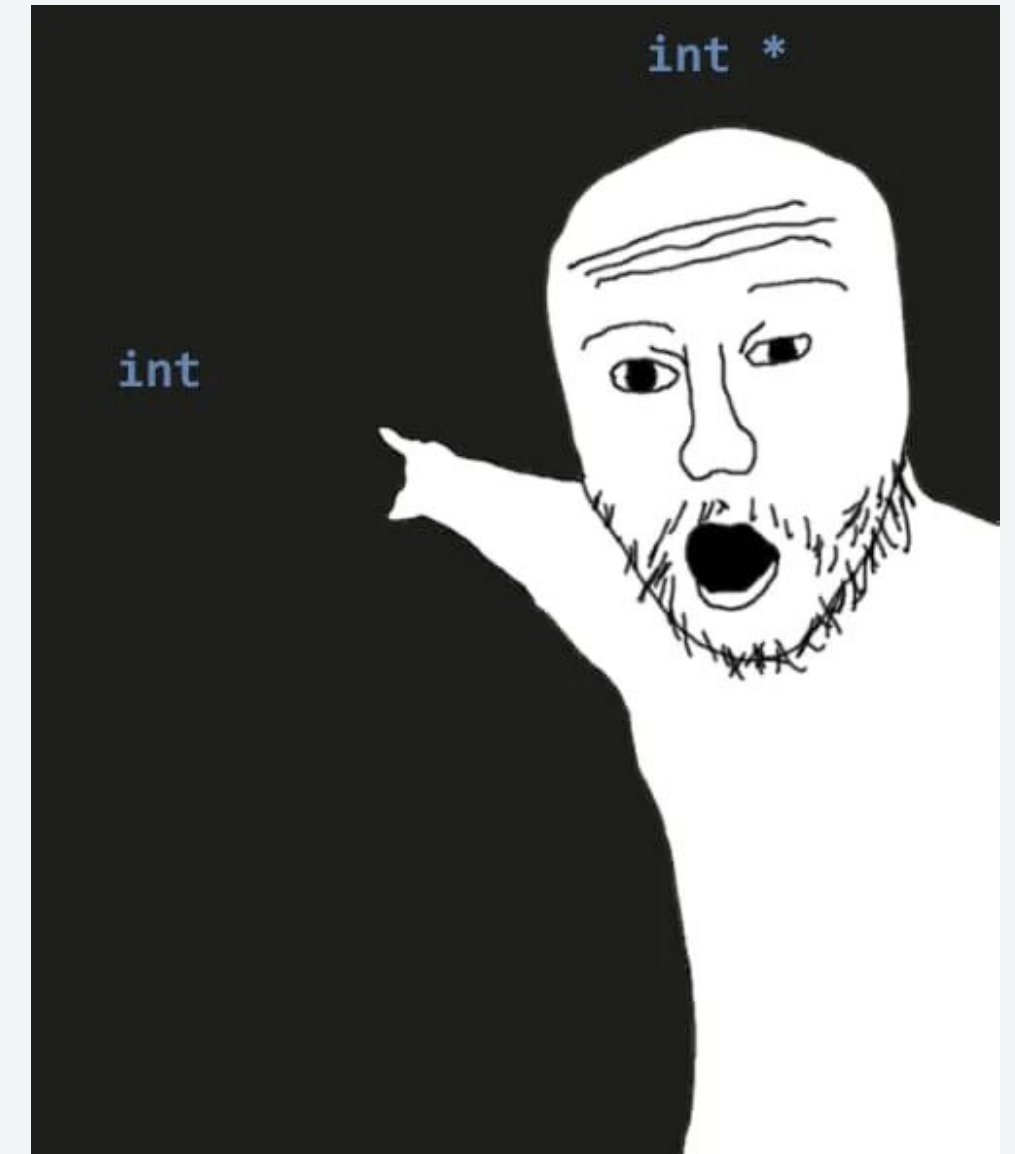
0x00000000

Address

0x00000000

0x00000004

- ptr is stored at 0x000000001
- ptr contains the address of m



Pointer Declaration, Initialization, and Usage

- Pointers are declared using the code shown below. Note that you have to specify the data type of the variable stored in the memory address.

```
<data type>* <variable_name>; /* Sample: int* ptr; */
```

Pointer Declaration, Initialization, and Usage

- Pointers are declared using the code shown below. Note that you have to specify the data type of the variable stored in the memory address.

```
<data type>* <variable_name>;  /* Sample: int* ptr; */
```

- Additionally, you can declare multiple pointers in one line or declare a pointer and a variable in one line.

```
int *ptr1, *ptr2;  /* ptr1 and ptr2 are both pointers */
```

```
int *ptr1, ptr2;   /* ptr1 - pointer, ptr2 - integer */
```

Pointer Declaration, Initialization, and Usage

- We use the **& operator** to **reference** the address (or get the address) of a variable.

```
int m = 5;           //4 bytes are allocated to m
int* ptr;            //declare a pointer to an integer
ptr = &m;             // reference "address of" m in ptr
```

Pointer Declaration, Initialization, and Usage

- We use the **& operator** to **reference** the address (or get the address) of a variable.

```
int m = 5;           //4 bytes are allocated to m
int* ptr;            //declare a pointer to an integer
ptr = &m;            // reference "address of" m in ptr
```

- We use the *** operator** for **dereferencing** the value (or getting the value) of a memory address through a pointer.

```
cout << *ptr << endl; // dereference the "value of"
                        address ptr
```

Pointer Declaration, Initialization, and Usage

- We use the **& operator** to **reference** the address (or get the address) of a variable.

```
int m = 5;           //4 bytes are allocated to m
int* ptr;            //declare a pointer to an integer
ptr = &m;            // reference "address of" m in ptr
```

- We use the *** operator** for **dereferencing** the value (or getting the value) of a memory address through a pointer.

```
cout << *ptr << endl; // dereference the "value of"
                        address ptr
```

We can also change the value stored in the address similar to how we change the value of a variable. But you have to keep in mind the correct operator.

Pointer Declaration, Initialization, and Usage

- Additionally, you have to keep these behaviors in mind.

```
int var = 121;
```

```
int* point_var = var;    // Wrong! point_var is an address but var is not
```

```
*point_var = &var;      // Wrong! &var is an address  
                        // *point_var is the value stored in &var
```

```
point_var = &var;        // Correct! point_var is an address and so is  
                        // &var
```

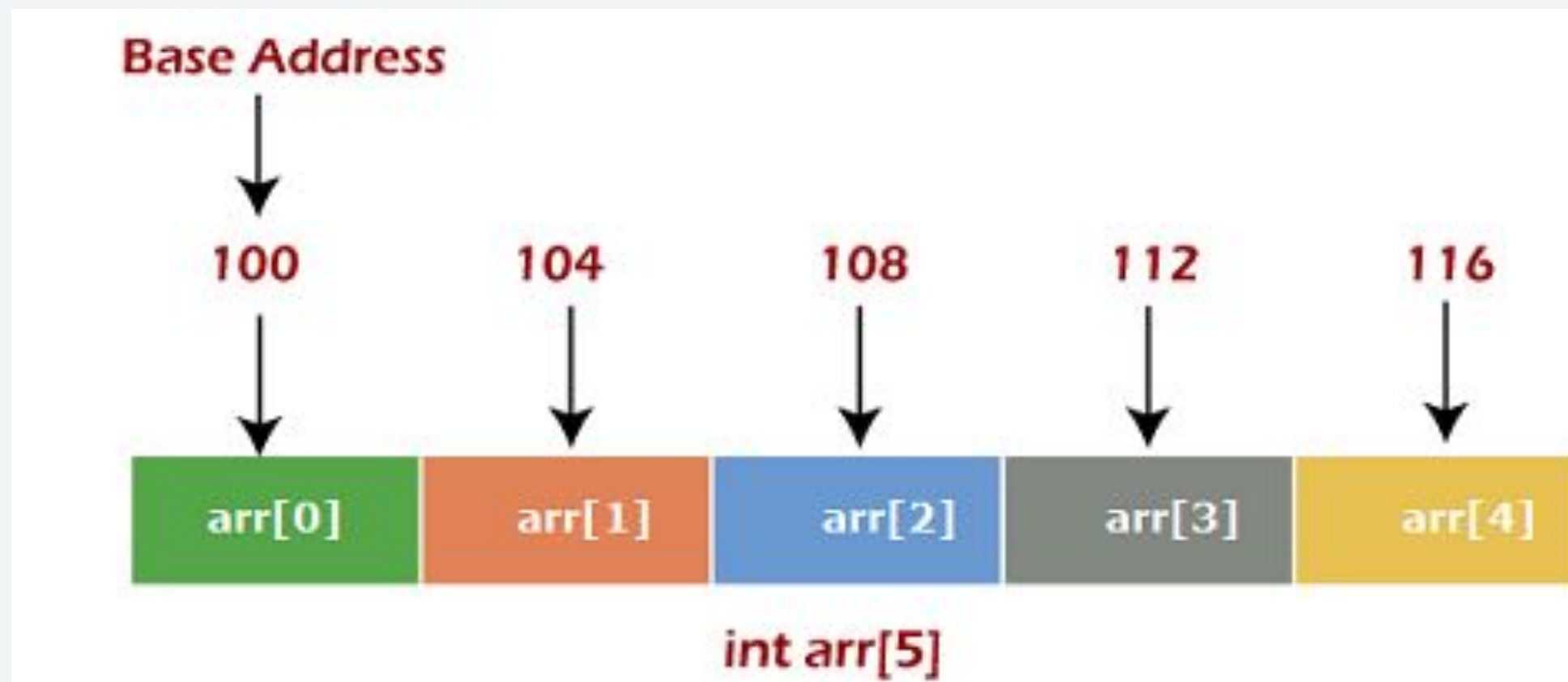
```
*point_var = var;        // Correct! both *point_var and var are values
```


Pointers and Arrays

- By definition, an array is a collection of elements stored together.

Pointers and Arrays

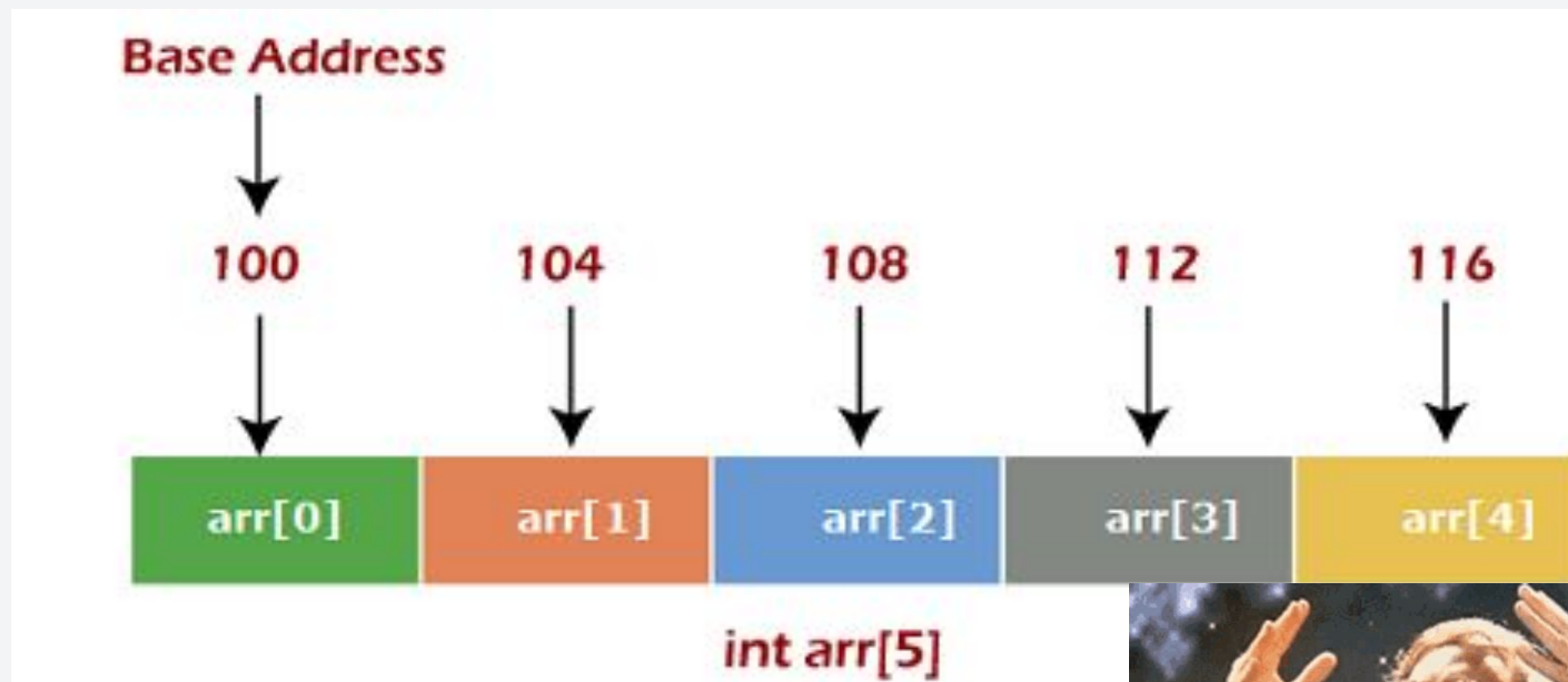
- By definition, an array is a collection of elements stored together.



- Elements of an array are stored in a contiguous manner.

Pointers and Arrays

- By definition, an array is a collection of elements stored together.

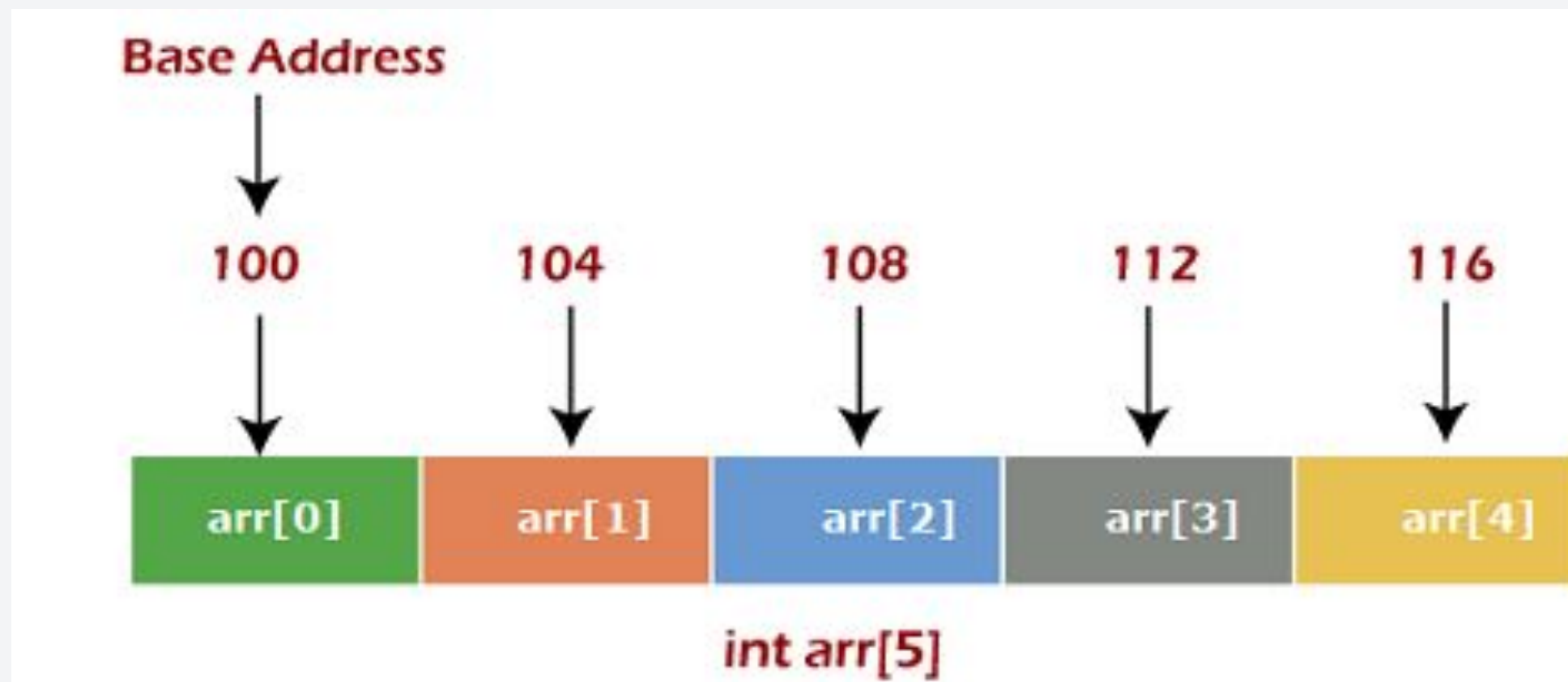


- Elements of an array are stored in a contiguous manner.
- `arr` is actually a pointer to the address of the start index



Pointers and Arrays

- By definition, an array is a collection of elements stored together.

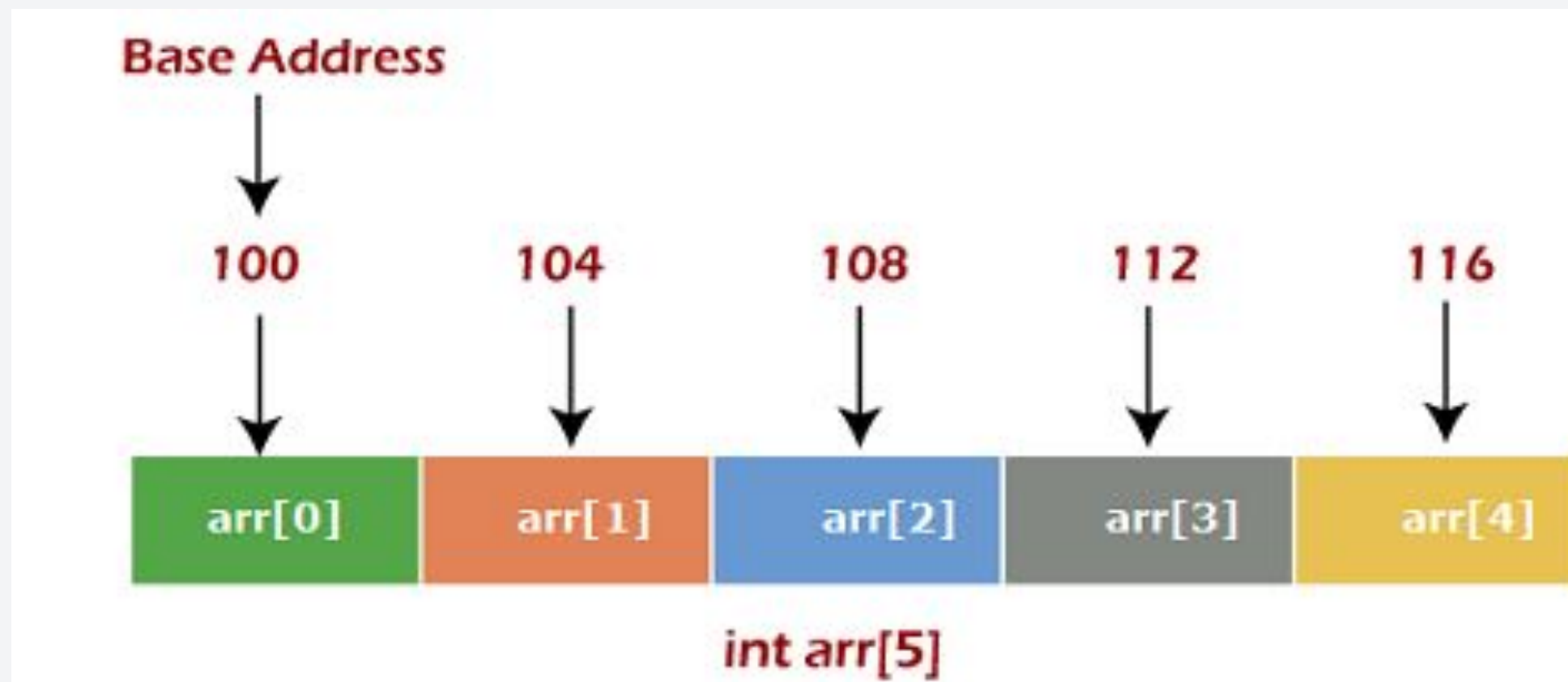


- Elements of an array are stored in a contiguous manner.
- `arr` is actually a pointer to the address of the start index

Question: Why are the base addresses labeled as 100, 104, 108, ...?

Pointers and Arrays

- By definition, an array is a collection of elements stored together.



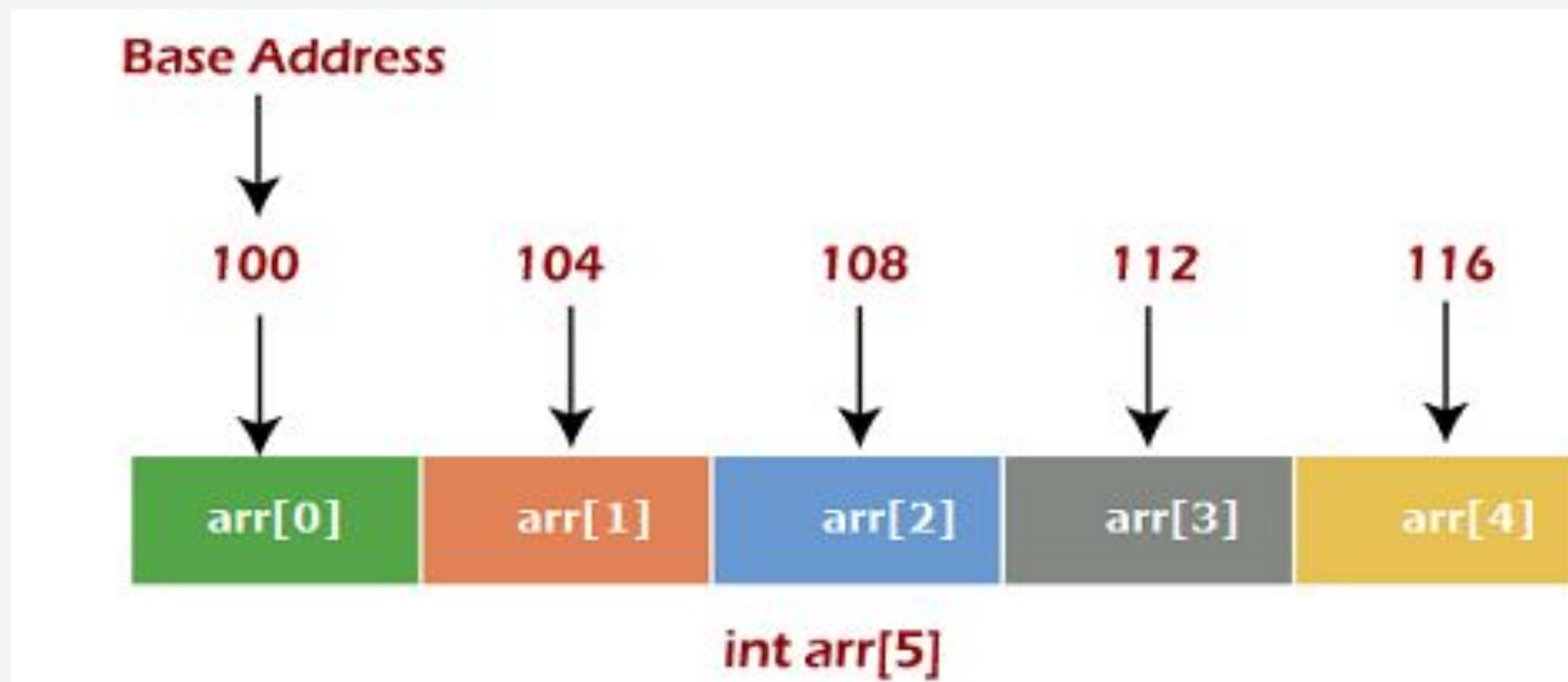
- Elements of an array are stored in a contiguous manner.
- `arr` is actually a pointer to the address of the start index

Question: Why are the base addresses labeled as 100, 104, 108, ...?

- An integer requires 4 bytes each.

Pointers and Arrays

- By definition, an array is a collection of elements stored together.



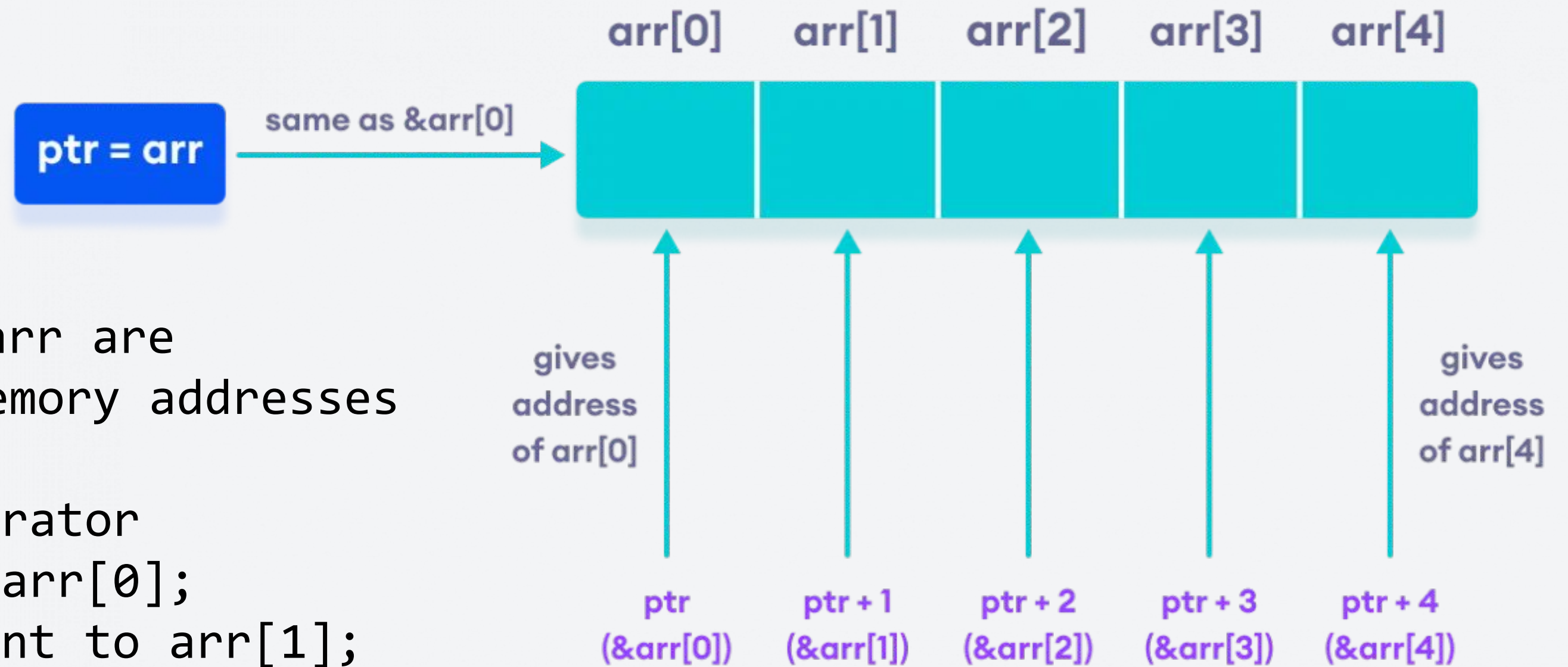
- Elements of an array are stored in a contiguous manner.
- `arr` is actually a pointer to the address of the start index

Since `arr` is a pointer, you can use it to access the values of the array using the dereferencing operator (`&`) and pointer arithmetic.

Arrays as Pointers and Pointer Arithmetic

```
int* ptr;  
int arr[5];  
ptr = arr; // ptr and arr are  
           // both memory addresses
```

```
// use dereference operator  
*ptr is equivalent to arr[0];  
*(ptr + 1) is equivalent to arr[1];  
*(ptr + 2) is equivalent to arr[2];  
*(ptr + 3) is equivalent to arr[3];  
*(ptr + 4) is equivalent to arr[4];
```



Arrays as Pointers and Pointer Arithmetic

Alternatively, you can also initialize ptr to the address of any element of the array. For example,

```
int* ptr;  
int arr[5];  
ptr = &arr[2];    // ptr and arr are  
                  // both memory addresses  
  
// use dereference operator  
*(ptr - 2) is equivalent to arr[0];  
*(ptr - 1) is equivalent to arr[1];  
*(ptr) is equivalent to arr[2];  
*(ptr + 1) is equivalent to arr[3];  
*(ptr + 2) is equivalent to arr[4];
```

Pass by Value vs Pass by Reference

Pass by Value

- Only the value of the variable is passed onto a function.
- Does not modify the value of the variable.

Pass by Reference

- The address or pointer to a variable is passed onto a function.
- Can change the original value of the variable.
- To pass a reference variable, apply the address or reference operator (&) to the variable name.

Pass by Value vs Pass by Reference - Analogy

Pass by Value

- Giving a person a photocopy of your birth certificate.
- They cannot modify your original birth certificate but they can modify the copy.
- Any changes to the copy does not affect the original.

Pass by Reference

- Telling the person the location of your birth certificate.
- This allows them to access the original copy of your birth certificate.
- They can modify your original birth certificate because they know the exact location of the original.

Pass by Value vs Pass by Reference - Usage

Pass by Value

```
#include <iostream>
int cubeByValue( int n ){
    n = n * n * n; // cube local variable n
    and return result
    return n;
}
int main( void ) {
    int number = 5; int result;
    cout << "The original value of number is"
    << number << endl;
    // pass number by value to cubeByValue
    result = cubeByValue(number);
    cout << "New value:" << number << endl;
}
```

Pass by Reference

```
#include <iostream>
void cubeByRef( int *n ){
    *n = (*n)*(*n)*(*n); // cube value pointed
    by pointer n
}
int main( void ) {
    int number = 5; int result;
    cout << "Original value: " << number <<
    endl;
    // pass number by reference to cubeByValue
    cubeByRef(&number);
    cout << "New value: " << number << endl;
}
```

Pass by Value vs Pass by Reference - Usage

Pass by Value

```
#include <iostream>
int cubeByValue( int n ){
    n = n * n * n; // cube local variable n
    and return result
    return n;
}
int main( void ) {
    int number = 5; int result;
    cout << "The original value of number is"
    << number << endl;
    // pass number by value to cubeByValue
    result = cubeByValue(number);
    cout << "New value:" << number << endl;
}
```

Pass by Reference

```
#include <iostream>
void cubeByRef( int *n ){
    *n = (*n)*(*n)*(*n); // cube value pointed
    by pointer n
}
int main( void ) {
    int number = 5; int result;
    cout << "Original value: " << number <<
    endl;
    // pass number by reference to cubeByValue
    cubeByRef(&number);
    cout << "New value: " << number << endl;
}
```

Pass by reference allows us to modify the actual content inside the memory address.

- Computer Memory
- Introduction to Pointers
- **Memory Management with Pointers**

Static vs Dynamic Memory Allocation

Static Memory Allocation

- Default allocation of memory in C++.
- Memory is allocated at compile time and the size must be known beforehand.
- Example:
 - Automatic Variables
 - Global and Static Variables

Dynamic Memory Allocation

- Allows developer to allocate memory during runtime using pointers.
- More flexible, but requires more management.
- Allocated memory should be freed to prevent **memory leak**.
- Method: new and delete

Dynamic Memory Allocation

- To dynamically allocate a new chunk of memory to a pointer, we use the `new` keyword.
 - We can use it to any data type and arrays.
- After we are finished with using the allocated memory space, we need to deallocate it to free up some memory using the `delete` keyword.
 - Deallocation of memory is essentially so that other programs/processes can use it.
 - Failure to do so will result in **memory leak**.

Static vs Dynamic Memory Allocation - Example

Static Memory Allocation

```
void example() {  
    int a = 5; // 'a' is statically allocated  
}  
  
int globalVar = 10; // Allocated globally
```

Dynamic Memory Allocation

```
int* ptr = new int;    // Allocates memory for  
one integer  
  
*ptr = 42;             // Assigns a value to  
the allocated memory  
  
int* arr = new int[5]; // Allocates an array  
of 5 integers  
  
delete ptr;            // Frees memory allocated for a  
single variable  
  
delete[] arr;          // Frees memory allocated for  
an array
```

Static vs Dynamic Memory Allocation - Example

Static Memory Allocation

```
void example() {  
    int a = 5; // 'a' is statically allocated  
}
```

```
int globalVar = 10; // Allocated globally
```

Use new to allocate memory to any data you want to store.

Dynamic Memory Allocation

```
int* ptr = new int; // Allocates memory for  
one integer
```

```
*ptr = 42; // Assigns a value to  
the allocated memory
```

```
int* arr = new int[5]; // Allocates an array  
of 5 integers
```

```
delete ptr; // Frees memory allocated for a  
single variable
```

```
delete[] arr; // Frees memory allocated for  
an array
```

Static vs Dynamic Memory Allocation - Example

Static Memory Allocation

```
void example() {  
    int a = 5; // 'a' is statically allocated  
}  
  
int globalVar = 10; // Allocated globally
```

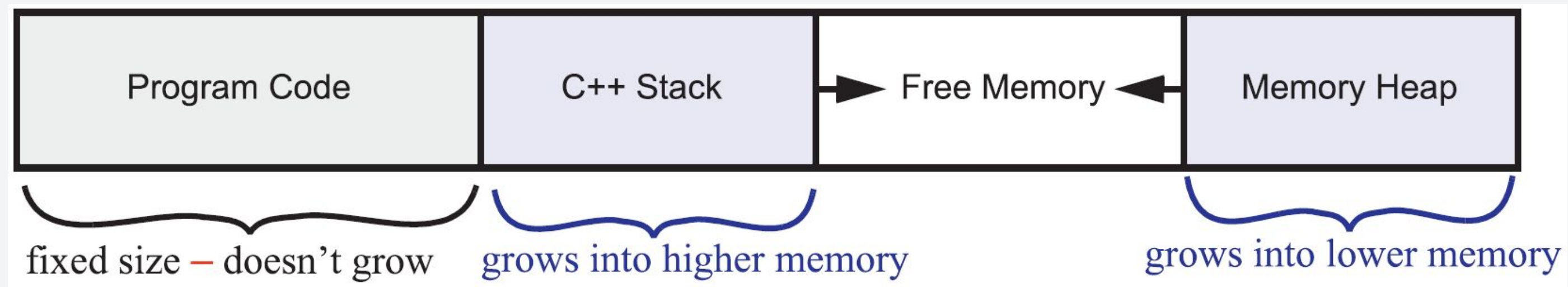
Very important to deallocate memory to prevent **memory leak**.

Dynamic Memory Allocation

```
int* ptr = new int;    // Allocates memory for  
one integer  
  
*ptr = 42;             // Assigns a value to  
the allocated memory  
  
int* arr = new int[5]; // Allocates an array  
of 5 integers  
  
delete ptr; // Frees memory allocated for a  
single variable  
delete[] arr; // Frees memory allocated for  
an array
```


Memory Layout in C++ Programs

- Sample Memory Layout in C++

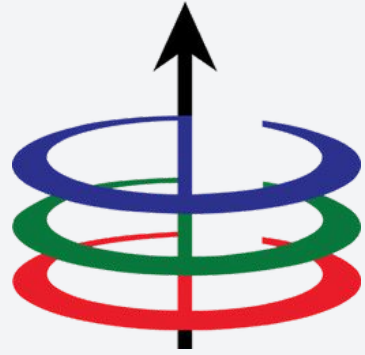


- The code itself is stored in memory. Additionally, there are two types of memory:
 - **Stack Memory** - portion of memory that stores and organizes function calls and static allocation of variables/arrays
 - **Heap Memory** - portion of memory which allows dynamic memory allocation (i.e. pointers which used new/delete keyword)

Any questions?

Ungraded Activity

Use dynamic memory allocation to initialize an **integer array with 6 elements {1,2,1,1,2,8}**. Additionally, create a pointer that points at the **3rd element** of the array. Lastly, iterate through the array and print each element **using pointer arithmetic**. Make sure to **deallocate the memory** after being used.



Lecture 1B

Pointers, Memory Management

EEE 121 2s2526

Steven S. Sison